



Fundamentos de Sistemas Embebidos

Facultad de Ingeniería

UNAM

Laboratorio de Fundamentos de Sistemas Embebidos

Profesor(a): Eduardo Fragoso Navarro

Asignatura: Fundamentos de Sistemas Embebidos

Grupo: 3

No de Práctica(s): 5

Integrante(s): Tepal Briseño Hansel Yael - 320078282

Ugartechea González Luis Antonio - 320223822

No. de brigada: 12

Semestre: 9°

Fecha de entrega: 6 de octubre de 2026

Observaciones: _____

CALIFICACIÓN: _____

Servidor web en el ESP32

Objetivos

El objetivo de esta práctica fue implementar un servidor web sobre la tarjeta XIAO ESP32-S3 para supervisar remotamente, mediante una conexión WiFi, la lectura de un potenciómetro y enviar mensajes a una pantalla OLED. Para ello se integraron la lectura analógica mediante el ADC1, la comunicación inalámbrica WiFi en modo punto de acceso (AP) y una comunicación bidireccional mediante WebSocket. También se consideró que el sistema puede operar en modo estación (STA), conectándose a una red WiFi existente.

También se buscó comprender la diferencia entre una petición HTTP convencional y una conexión persistente WebSocket. La página se entrega inicialmente mediante HTTP y, una vez cargada, recibe automáticamente las mediciones del ADC por la ruta /ws; de forma inversa, el texto escrito en el navegador se transmite por esa misma conexión para visualizarse en la OLED.

Resultados y análisis

Ejercicio 1: Circuito

Se armó el circuito con un potenciómetro y una pantalla OLED conectada al bus I2C de la tarjeta. El potenciómetro se conectó al pin D5, correspondiente al GPIO 6 y al canal 5 del ADC1. La pantalla utilizó D8 (GPIO 7) como SDA y D7 (GPIO 44) como SCL. En el programa se configuró el ADC con una resolución de 12 bits, por lo que las lecturas se encuentran en el intervalo de 0 a 4095.



Figure 1: Circuito implementado y pantalla OLED con el servidor activo.

La pantalla OLED se controló mediante la biblioteca U8g2 y una implementación de las funciones de transferencia I2C. Esto permitió actualizar en la misma pantalla el estado del servidor, el mensaje recibido y el valor actual del potenciómetro.

Ejercicio 2: Funcionamiento del sistema

La implementación inalámbrica se realizó por ambos métodos mediante WiFi y modo de red AP (punto de acceso). Por cuestiones de facilidad de configuración y estabilidad durante las pruebas, el ESP32 creó la red ESP32-S3 con la contraseña 12345678, mientras que el navegador se conectó directamente a la dirección 192.168.4.1. Esta alternativa permitió realizar la práctica sin depender de un router externo. La evidencia de la terminal confirma la creación del punto de acceso y la dirección que debe abrirse en el navegador.

```
I (874) wifi_init: WiFi RX IRAM OP enabled
¡Punto de Acceso creado! Conéctate a la red: ESP32-S3 y abre http://192.168.4.1
I (884) phy_init: phy_version 712,87e8c20e, Apr 13 2026, 18:51:10
```

Figure 2: Creación del punto de acceso y dirección IP del ESP32.

```
lethalsopaper@phantom [~]$ nmcli device wifi connect ESP32-S3 password 12345678
Device 'wlan0' successfully activated with '80a5f321-fdc1-4c0d-b6f3-883de88582cf'.
```

Figure 3: Conexión de la computadora a la red inalámbrica del ESP32.

La modalidad de conexión se selecciona en el código mediante la constante `WIFI_MODE_SELECTION`. El valor cero selecciona el modo estación (STA), mientras que un valor diferente de cero selecciona el modo punto de acceso (AP). De esta forma, ambas configuraciones utilizan el mismo servidor web y solo cambia la forma en que se establece la red WiFi.

```
1 #define WIFI_MODE_SELECTION 1
2
3 #if WIFI_MODE_SELECTION == 0
4     // Modo estacion: conectarse a una red WiFi existente
5 #else
6     // Modo AP: crear una red WiFi propia
7 #endif
```

Listing 1: Selección entre WiFi STA y punto de acceso AP

Cuando se selecciona STA, el ESP32 crea una interfaz de estación, registra los eventos de WiFi y configura el SSID y la contraseña del router. Después de iniciar el periférico, el evento `WIFI_EVENT_STA_START` solicita la conexión y, al recibirse `IP_EVENT_STA_GOT_IP`, se imprime la dirección asignada por el router. El navegador debe conectarse a esa misma red y abrir dicha dirección.

```
1 esp_netif_create_default_wifi_sta();
2 wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
3 esp_wifi_init(&cfg);
4
```

```

5  wifi_config_t wifi_config = {
6      .sta = {
7          .ssid = WIFI_SSID,
8          .password = WIFI_PASS,
9      },
10 };
11
12 esp_wifi_set_mode(WIFI_MODE_STA);
13 esp_wifi_set_config(WIFI_IF_STA, &wifi_config);
14 esp_wifi_start();

```

Listing 2: Configuración de la conexión WiFi en modo STA

En modo AP, el ESP32 crea su propia red inalámbrica en lugar de buscar un router. Se configura el nombre de la red, la contraseña, el número máximo de clientes y el método de autenticación. Al iniciar WiFi, la interfaz utiliza la dirección predeterminada 192.168.4.1; por ello el navegador se conecta a la red ESP32-S3 y solicita la página mediante esa dirección.

```

1  esp_netif_create_default_wifi_ap();
2  wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
3  esp_wifi_init(&cfg);
4
5  wifi_config_t wifi_config = {
6      .ap = {
7          .ssid = AP_SSID,
8          .ssid_len = strlen(AP_SSID),
9          .password = AP_PASS,
10         .max_connection = 4,
11         .authmode = WIFI_AUTH_WPA2_PSK,
12     },
13 };
14
15 esp_wifi_set_mode(WIFI_MODE_AP);
16 esp_wifi_set_config(WIFI_IF_AP, &wifi_config);
17 esp_wifi_start();

```

Listing 3: Configuración del punto de acceso WiFi

En modo AP, la función de inicialización configura el nombre de la red, la contraseña y el número máximo de clientes. Si se utilizara el modo STA, sería necesario crear una interfaz de estación, indicar el SSID y la contraseña del router, esperar el evento de obtención de IP y abrir en el navegador la dirección asignada por dicho router. El resto del servidor HTTP y de la página podría conservarse prácticamente sin cambios.

Análisis de la conexión WiFi. La selección de AP fue importante porque permitió crear una red autónoma, fácil de configurar y estable durante las pruebas. En una aplicación conectada a una infraestructura existente, STA sería más conveniente porque permitiría que el ESP32 y otros dispositivos compartieran la red del router.

El servidor atiende dos rutas. La ruta / devuelve la página HTML embebida en el firmware con tipo text/html. La ruta /ws se registra como WebSocket y permite la comunicación en ambos sentidos. La página abre la conexión con:

```
1 var ws = new WebSocket('ws://' + location.host + '/ws');
2
3 ws.onmessage = function(event) {
4     document.getElementById('adc_val').innerText = event.data;
5 };
6
7 function enviarMensaje() {
8     var msg = document.getElementById('mensaje').value;
9     if (ws.readyState === WebSocket.OPEN) {
10         ws.send(msg);
11     }
12 }
```

Listing 4: Apertura de la conexión WebSocket en el navegador

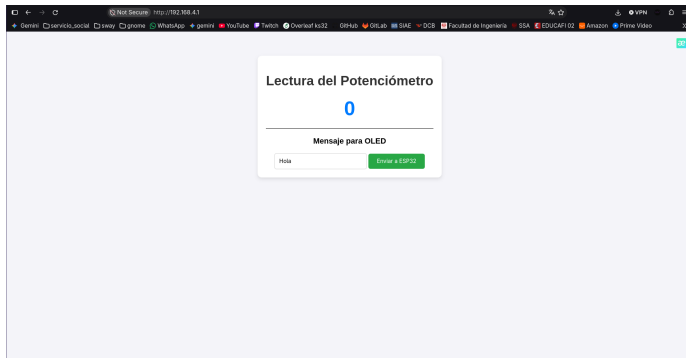
La tarea tarea_adc realiza una lectura cada 500 ms. Cuando existe un cliente conectado, obtiene el valor del ADC, actualiza la OLED y envía el valor como una trama de texto mediante WebSocket. El periodo elegido permite observar cambios del potenciómetro sin generar una cantidad excesiva de tráfico. Si se redujera demasiado, aumentarían la frecuencia de ejecución de la tarea, el tráfico TCP y el trabajo de actualización del navegador; con varios dispositivos conectados, el mismo dato tendría que enviarse a cada cliente.

```
1 adc_oneshot_read(adc1_handle, ADC_CHANNEL_5, &adc_raw);
2 snprintf(buffer_adc, sizeof(buffer_adc), "%d", adc_raw);
3 mostrar_adc_oled(adc_raw);
4 ws_pkt.payload = (uint8_t *)buffer_adc;
5 ws_pkt.len = strlen(buffer_adc);
6 httpd_ws_send_frame_async(servidor_global, fds[i], &ws_pkt);
7 vTaskDelay(pdMS_TO_TICKS(500));
```

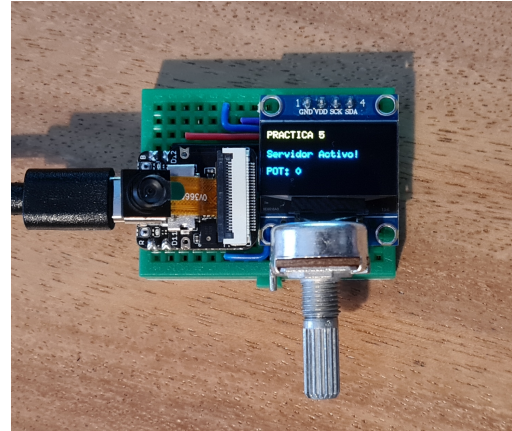
Listing 5: Lectura periódica y envío del valor del ADC

Análisis de la lectura del ADC. El envío periódico por WebSocket es importante porque el ESP32 empuja la medición sin que el navegador tenga que realizar solicitudes repetitivas. El periodo de 500 ms ofrece una actualización suficientemente rápida para observar el potenciómetro y, al mismo tiempo, limita el tráfico y el consumo de recursos. Con varios clientes, la lectura debe enviarse a cada conexión, por lo que aumentar demasiado la frecuencia elevaría la carga del servidor.

En la siguiente imagen se muestra el como se inicializa la página web una vez es ingresada la dirección del ESP32 en el navegador. La pantalla OLED indica que el servidor está activo y listo para recibir mensajes. En este caso indica el valor 0 por que se dejó en el valor mínimo para la evidencia.



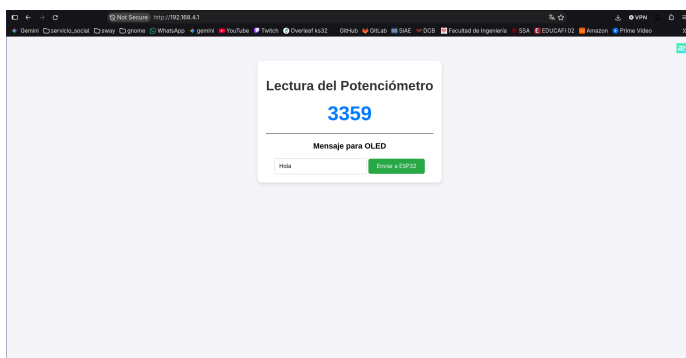
(a) Página web al iniciar.



(b) OLED asociada.

Figure 4: Conexión inicial y servidor activo.

En la siguiente figura se muestra la actualización de la lectura del potenciómetro. La página web muestra el valor 3359 y la pantalla OLED muestra el valor 3347, esto debido a que la lectura del ADC se realiza cada 500 ms y puede variar ligeramente entre una lectura y otra. Cabe destacar que hasta este momento aún no se ha enviado ningún mensaje desde la página web, por lo que la OLED solo muestra el valor del potenciómetro.



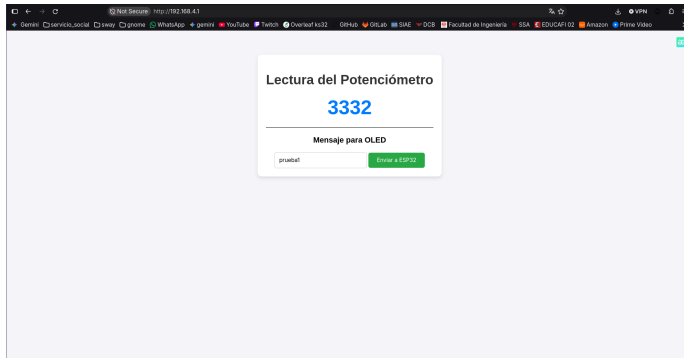
(a) Lectura mostrada: 3359.



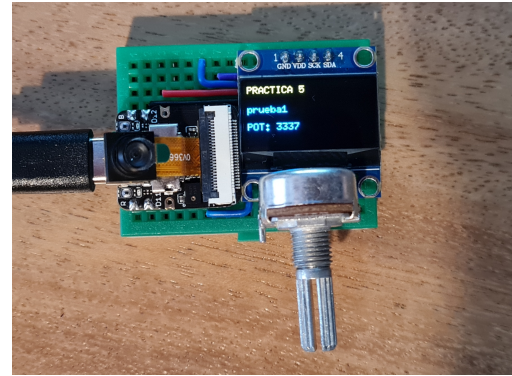
(b) Lectura en la OLED: 3347.

Figure 5: Actualización de la lectura del potenciómetro.

Finalmente, en la siguiente figura se muestra la página web con el mensaje prueba1 escrito en el campo de texto y la pantalla OLED mostrando el mismo mensaje junto con la lectura del potenciómetro. Esto demuestra que la comunicación bidireccional mediante WebSocket funciona correctamente, permitiendo enviar mensajes desde el navegador al ESP32 y viceversa.



(a) Mensaje escrito en la página: prueba1.



(b) Mensaje mostrado en la OLED junto con la lectura del potenciómetro.

Figure 6: Envío de mensaje desde el navegador al ESP32 mediante WebSocket.

Análisis del envío de mensajes. La recepción de tramas de longitud desconocida y la reserva de $len + 1$ bytes son importantes para evitar desbordamientos y garantizar que el texto sea una cadena C válida. La conexión WebSocket también permite que la página envíe datos al ESP32 sin abrir otra solicitud HTTP. Como la página usa UTF-8, los acentos y la letra ñ llegan como varios bytes; para mostrarlos correctamente sería necesario utilizar una fuente de la OLED que contenga esos caracteres.

Análisis de los protocolos. HTTP resulta adecuado para entregar la página inicial, mientras que WebSocket mantiene un canal TCP persistente, bidireccional y confiable para las mediciones y los mensajes. UDP tendría menor sobrecarga y latencia, pero no garantiza entrega ni orden, y el navegador no puede abrir directamente sockets UDP arbitrarios desde JavaScript. Por ello, WebSocket fue la opción más apropiada para esta interfaz web.

Conclusiones

La práctica permitió integrar adquisición analógica, conectividad WiFi, HTTP, WebSocket y una pantalla OLED. El ESP32 creó la red AP, entregó la página, actualizó el valor del potenciómetro y recibió mensajes desde el navegador.

HTTP es adecuado para entregar la página inicial; WebSocket mantiene una conexión TCP persistente y bidireccional para enviar mediciones y mensajes. UDP tendría menor sobrecarga, pero no garantiza entrega ni orden y no puede utilizarse directamente desde JavaScript en el navegador.

Como cualquier dispositivo de la red AP podría enviar texto, una aplicación real debería incorporar autenticación, cifrado, validación de mensajes y control de acceso. Esta solución puede aplicarse en monitoreo ambiental, automatización doméstica, mantenimiento y control remoto de actuadores.

NOTA: Si se desea consultar el código fuente completo del proyecto, se puede acceder al repositorio de GitHub en la siguiente dirección: https://github.com/lethalSopape/r/laboratorio_sistemas_embebidos